

view_interface::[c]rbegin()

- - Abstract
 - Revision history
 - Discussion
 - Design
 - Implementation experience
 - Proposed change

Abstract

This paper provides `rbegin()/rend()/crbegin()/crend()` methods to `ranges::view_interface`, making its interface more symmetric with the existing `cbegin()/cend()` members and enhancing convenience for views.

Revision history

R0

Initial revision.

Discussion

In C++23, `view_interface` added `cbegin()/cend()` members to improve const-correctness and interface consistency. However, it still lacks the corresponding reverse iterator members `rbegin()/rend()/crbegin()/crend()`, which are extremely common in the library:

Reversible Ranges	.rbegin()	.rend()	.crbegin()	.crend()
array/vector/inplace_vector	✓	✓	✓	✓
deque/list/hive	✓	✓	✓	✓
set/multiset/map/multimap	✓	✓	✓	✓
flat_set/flat_multiset/flat_map/flat_multimap	✓	✓	✓	✓
span/string/string_view	✓	✓	✓	✓
stacktrace	✓	✓	✓	✓
ranges::meow_view	✗	✗	✗	✗

Although `back()` is provided for bidirectional views, accessing the second-to-last element or performing general reverse iteration remains inconvenient without these members. Users of custom views are therefore forced to rely on the free functions `ranges::rbegin(r)` or wrap the view with `views::reverse`:

```
auto my_view = some-view;

// today
for (auto it = ranges::rbegin(my_view); it != ranges::rend(my_view); ++it) {
    // do some work
}
```

```

}

for (auto&& elem : my_view | views::reverse) {
    // do some work
}

// expected, but always not valid
for (auto it = my_view.rbegin(); it != my_view.rend(); ++it) {
    // do some work
}

```

`ranges::rbegin` is a CPO with non-trivial lookup rules, which introduces cognitive overhead compared to simple member access. The use of `views::reverse` introduces an additional adaptor layer and may require moving the underlying range for move-only views.

Moreover, the lack of `rbegin`/`rend` members imposes an unnecessary syntactic overhead when interfacing with legacy APIs:

```

template<class BidirectionalIterator>
void legacy_algo(I first, I last);

template<bidirectional_range R>
void my_algo(R&& r) {
    auto my_view = some-view;

    legacy_algo(ranges::rbegin(my_view), ranges::rend(my_view));

    auto reversed = my_view | views::reverse;
    legacy_algo(reversed.begin(), reversed.end());

    // expected, but always not valid
    legacy_algo(my_view.rbegin(), my_view.rend());
}

```

It is worth noting that the following will always work:

```

// always valid for input ranges
for (auto it = my_view.cbegin(); it != my_view.cend(); ++it) {
    // do some work
}

```

In the current standard, it is an established design philosophy that if a container provides `cbegin` member and supports bidirectional iteration, it consistently provides `rbegin` and `crbegin` members. Types like `forward_list` or `unordered_map` omit `rbegin` because they are not bidirectional; while types like `simd::vec` are not common-range because their `end` returns a simple `default_sentinel`, and therefore only provide `cbegin` member.

Currently, `view_interface` is the only major entity that fails to provide these members even when all prerequisites - bidirectional iteration and the common range property — are met. This omission is an inconsistency in the library's design, as views should ideally mirror the interface of the ranges they represent to ensure a predictable developer experience.

Adding these commonly expected members allows users to write more intuitive and natural code, providing a more symmetric and consistent interface for views while imposing no additional burden on view authors.

Design

Constraint for `rbegin()`/`rend()`/`crbegin()`/`crend()`

Those members are provided only when the derived view models both `bidirectional_range` and `common_range`. This matches the existing constraint on `back()` and ensures that the end iterator can be extracted in constant time.

The corresponding `const` version has the same constraint because it is equivalent to `make_reverse_iterator(cbegin())` or `make_const_iterator(rbegin())`, and both `cbegin()` and `make_const_iterator` simply requires `input_range` which has been accommodated by `bidirectional_range`.

Specialization for `reverse_iterator`

To prevent unnecessary template nesting, the proposed `rbegin()` and `crbegin()` will unwrap the underlying iterator by calling `.base()` if it is already a specialization of `reverse_iterator`. This approach mirrors the existing behavior of `views::reverse`, which avoids creating double-reversed types by returning the original base range.

This design leads to the `ranges::rbegin(r)` and `ranges::crbegin(r)` naturally benefits from this optimization, as it prioritizes calling the member function when available:

Before	After P4179
<pre>// A range with an iterator of type reverse_iterator ranges::subrange<reverse_iterator<int*>, reverse_iterator<int*>> s;</pre>	
<pre>// invalid s.rbegin();</pre>	<pre>// int* s.rbegin();</pre>
<pre>// invalid std::rbegin(s);</pre>	<pre>// int* std::rbegin(s);</pre>
<pre>// reverse_iterator<reverse_iterator<int*>> ranges::rbegin(s);</pre>	<pre>// int* ranges::rbegin(s);</pre>
<pre>auto r = s views::reverse; // int* r.begin();</pre>	<pre>auto r = s views::reverse; // int* r.begin();</pre>
Const version	
<pre>// invalid s.crbegin();</pre>	<pre>// basic_const_iterator<int*> s.crbegin();</pre>
<pre>// invalid std::crbegin(s);</pre>	<pre>// int* std::crbegin(s);</pre>
<pre>// basic_const_iterator< reverse_iterator< reverse_iterator<int*>>> ranges::crbegin(s);</pre>	<pre>// basic_const_iterator<int*> ranges::crbegin(s);</pre>
<pre>auto r = s views::as_const views::reverse; // reverse_iterator< basic_const_iterator< reverse_iterator<int*>>> r.begin();</pre>	<pre>auto r = s views::as_const views::reverse; // reverse_iterator< basic_const_iterator< reverse_iterator<int*>>> r.begin();</pre>
<pre>auto r = s views::reverse views::as_const; // basic_const_iterator<int*> r.begin();</pre>	<pre>auto r = s views::reverse views::as_const; // basic_const_iterator<int*> r.begin();</pre>

Implementation experience

The author implemented those members based on `libstdc++`.

See [godbolt link](#) for details.

Proposed change

1. Add a new feature-test macro to 17.3.2 [\[version.syn\]](#):

```
#define __cpp_lib_view_interface 2026XXL // also in <ranges>
```

2. Modify 26.5.3 [\[view.interface\]](#) as indicated:

```
namespace std::ranges {
  template<class D>
    requires is_class_v<D> && same_as<D, remove_cv_t<D>>
    class view_interface {
    [...]
    public:
    [...]

    constexpr auto cbegin() requires input_range<D> {
      return ranges::cbegin(derived());
    }
  };
}
```

```

    }
    constexpr auto cbegin() const requires input_range<const D> {
        return ranges::cbegin(derived());
    }
    constexpr auto cend() requires input_range<D> {
        return ranges::cend(derived());
    }
    constexpr auto cend() const requires input_range<const D> {
        return ranges::cend(derived());
    }
}

constexpr auto rbegin()
    requires bidirectional_range<D> && common_range<D>;
constexpr auto rbegin() const
    requires bidirectional_range<const D> && common_range<const D>;
constexpr auto rend()
    requires bidirectional_range<D> && common_range<D>;
constexpr auto rend() const
    requires bidirectional_range<const D> && common_range<const D>;

constexpr auto crbegin()
    requires bidirectional_range<D> && common_range<D>;
constexpr auto crbegin() const
    requires bidirectional_range<const D> && common_range<const D>;
constexpr auto crend()
    requires bidirectional_range<D> && common_range<D>;
constexpr auto crend() const
    requires bidirectional_range<const D> && common_range<const D>;
};
}

```

[...]

```

constexpr auto rbegin()
    requires bidirectional_range<D> && common_range<D>;
constexpr auto rbegin() const
    requires bidirectional_range<const D> && common_range<const D>;

```

-?- Let T be decltype(ranges::end(derived())).

-?- Returns:

(?.?) - If T is a specialization of reverse_iterator, ranges::end(derived()).base().
(?.?) - Otherwise, reverse_iterator(ranges::end(derived())).

```

constexpr auto rend()
    requires bidirectional_range<D> && common_range<D>;
constexpr auto rend() const
    requires bidirectional_range<const D> && common_range<const D>;

```

-?- Let T be decltype(ranges::begin(derived())).

-?- Returns:

(?.?) - If T is a specialization of reverse_iterator, ranges::begin(derived()).base().
(?.?) - Otherwise, reverse_iterator(ranges::begin(derived())).

```

constexpr auto crbegin()
    requires bidirectional_range<D> && common_range<D>;
constexpr auto crbegin() const
    requires bidirectional_range<const D> && common_range<const D>;

```

-?- Returns: make_const_iterator(possibly-const-range(derived()).rbegin()).

```

constexpr auto crend()
    requires bidirectional_range<D> && common_range<D>;
constexpr auto crend() const
    requires bidirectional_range<const D> && common_range<const D>;

```

-?- Returns: make_const_iterator(possibly-const-range(derived()).rend()).